

SHIFT REDUCE PARSERS

→ Shift reduce parsing is the process of reducing a string to the start symbol of a grammar.

A string $\xrightarrow{\text{Reduce to}}$ Starting Symbol

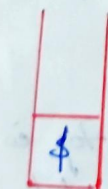
→ This reduction can be achieved by directly handling the rightmost derivations from the starting symbol to the input symbol.

→ Two data structures are required to implement a shift reduce parser

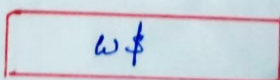
- * An input buffer to hold the input string
- * A stack to keep the grammar symbol for accessing production rules.

Shift-Reduce Parsing Algorithm

→ Initially, the stack contains the $\$$ symbol and the input buffer contains the input string with the $\$$ symbol at its end.



Stack



Input Buffer

→ The parser works by

- * Moving the input symbol on the top of the stack (Shift operation)
- * Until a handle appears on the top of the stack. (Reduce operation)



Stack



Input Buffer

→ Repeat the above step until

- * An error is detected (or)
- * The string is accepted. (The stack contains only the start symbol and input buffer becomes empty).

→ If we achieve the above configuration, the parsing will be completed successfully.

Basic Operations

1. Shift:

→ Moving current symbol from input buffer onto the stack.

2. Reduce:

→ Handle appearing on the stack top is replaced with appropriate non-terminal symbol.

(RHS of the production rule is popped out of stack and LHS of production rule is pushed in stack)

3. Accept

→ If the stack contains the starting symbol of the input string and input buffer is empty, the input string is accepted.

4. Error

→ The parser can neither perform shift action or reduce action and not even accept action.

Rules

1. If the priority of the incoming operator is higher than the operators priority at the top of the stack, then perform shift action.

2. If the priority of the incoming operator is equal or less than the operators priority at the top of the stack, then perform reduce action.

1. Consider the following grammar

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

$$\begin{aligned} E &\Rightarrow^m E + E \\ &\Rightarrow^m E + E * E \\ &\Rightarrow^m E + E * id \\ &\Rightarrow^m E + id * id \\ &\Rightarrow^m id + id * id \end{aligned}$$

Perform shift reduce parsing for the input string
 $id + id * id$

Solution:

The priority order of operators: $id > * > + > E > \$$

Stack	Input Buffer	Parsing Action	
\$	$id + id * id \$$	Shift	$id > \$$ (Rule 1)
$\$ id$	$+ id * id \$$	Reduce by $E \rightarrow id$	$+ < id$ (Rule 2)
$\$ E$	$+ id * id \$$	Shift	$\# + > E$ (Rule 1)
$\$ E +$	$id * id \$$	Shift	$id > +$ (Rule 1)
$\$ E + id$	$* id \$$	Reduce by $E \rightarrow id$	$* < id$ (Rule 2)
$\$ E + E$	$* id \$$	Shift	$* > E$ (Rule 1)
$\$ E + E *$	$id \$$	Shift	$id > *$ (Rule 1)
$\$ E + E * id$	$\$$	Reduce by $E \rightarrow id$	$\$ < id$ (Rule 2)
$\$ E + E * E$	$\$$	Reduce by $E \rightarrow E * E$	$\$ < E$ (Rule 2)
$\$ E + E$	$\$$	Reduce by $E \rightarrow E + E$	$\$ < E$ (Rule 2)
$\$ E$	$\$$	Accept	

2. Consider the following grammar

$$S \rightarrow S + S$$

$$S \rightarrow S - S$$

$$S \rightarrow (S)$$

$$S \rightarrow a$$

$$\begin{aligned} S &\Rightarrow^m S - \underline{S} \\ &\Rightarrow^m S - (S) \\ &\Rightarrow^m S - (S + S) \\ &\Rightarrow^m S - (S + a) \\ &\Rightarrow^m \underline{S} - (a + a) \\ &\Rightarrow^m a - (a + a) \end{aligned}$$

Perform shift reduce parsing for the input string $a_1 - (a_2 + a_3)$

Solution

Priority order: $a_1/a_2/a_3 > () > + / - > S > \$$

Stack	Input Buffer	Parsing Action
\$	$a_1 - (a_2 + a_3) \$$	Shift a_1
$\$ a_1$	$- (a_2 + a_3) \$$	Reduce by $S \rightarrow a$
$\$ S$	$- (a_2 + a_3) \$$	Shift -
$\$ S -$	$(a_2 + a_3) \$$	Shift (
$\$ S - ($	$a_2 + a_3) \$$	Shift a_2
$\$ S - (a_2$	$+ a_3) \$$	Reduce by $S \rightarrow a$
$\$ S - (S$	$+ a_3) \$$	Shift +
$\$ S - (S +$	$a_3) \$$	Shift a_3
$\$ S - (S + a_3$	$) \$$	Reduce by $S \rightarrow a$
$\$ S - (S + S$	$) \$$	Shift)
$\$ S - (S + S)$	$\$$	Reduce by $S \rightarrow S + S$
$\$ S - (S)$	$\$$	Reduce by $S \rightarrow (S)$
$\$ S - S$	$\$$	Reduce by $S \rightarrow S - S$
$\$ S$	$\$$	Accept